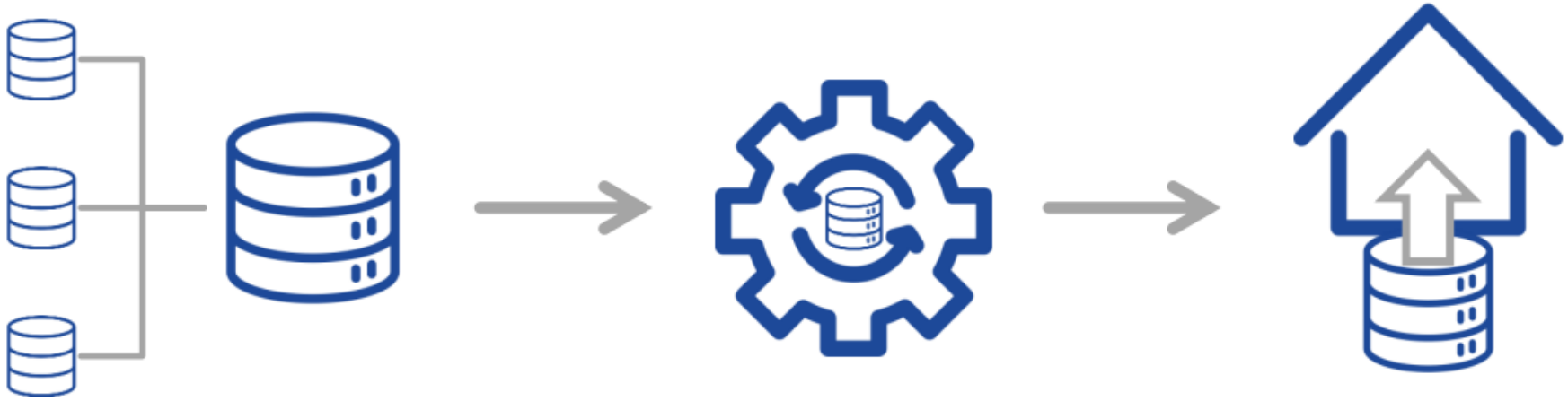
Arquitecturas ETL y Modelos de IA Aplicados

INSTITUCIÓN UNIVERSITARIA

SNIES 4709

Especialización en Analítica de Datos Aplicada a los Negocios
Sesión 2

ETL | Extract Transform Load



Extract
Recolecta y valida los datos de diferentes fuentes

Transform
Procesa y organiza los datos extraídos para convertirlos en útiles

Load
Mueve los datos transformados a un repositorio

ETL | Tipos de Extracciones

Carga Total

- Carga inicial
- Tablas fijas
- Migraciones, datasets históricos, ...

Carga incremental

- Datos nuevos desde la última carga
- Columna de partición
- Cargas diarias, semanales, ...

Carga en tiempo real

- Los datos se procesan y almacenan en cuanto están disponibles
- Herramientas específicas

ETL | Tipos de Transformaciones

Limpieza de Datos

- Eliminación valores nulos
- Eliminación duplicados

Filtrado

- Filtrado por reglas
- Filtrado por tiempo

Unificación

- Conversión de tipos
- Mapeo de columnas

Operaciones

- Agregaciones
- Normalización
- Estandarización

Anonimización

- Protección de datos sensibles

ETL | Tipos de Cargas (Load)

Base datos relacionales
- Datos estructurados
- MySQL, PostgreSQL

Base datos NoSQL
- Datos semi-estructurados o no estructurados
- Clave-valor (Redis)
- Documentos (MongoDB)
- Grafos (Neo4j)

Data Lakes
- Grandes volúmenes de datos en forma “cruda”
- AWS S3, Azure Data Lake

Data Warehouse
- Grandes volúmenes de datos listos para consulta
- Snowflake, Google BigQuery

Almacenamiento distribuido
- Sistemas a gran escala
- HDFS

Delta Lakes | Una solución perfecta para ETL

Capa de almacenamiento open-source nativa en Databricks sobre Apache Spark basado en Parquet y logs de transacciones

Transacciones ACID

Schema enforcement

Time Travel

Optimización de datos

Transacciones ACID| La base de la integridad en el Data Lake

Atomicity

Consistency

Isolation

Durability

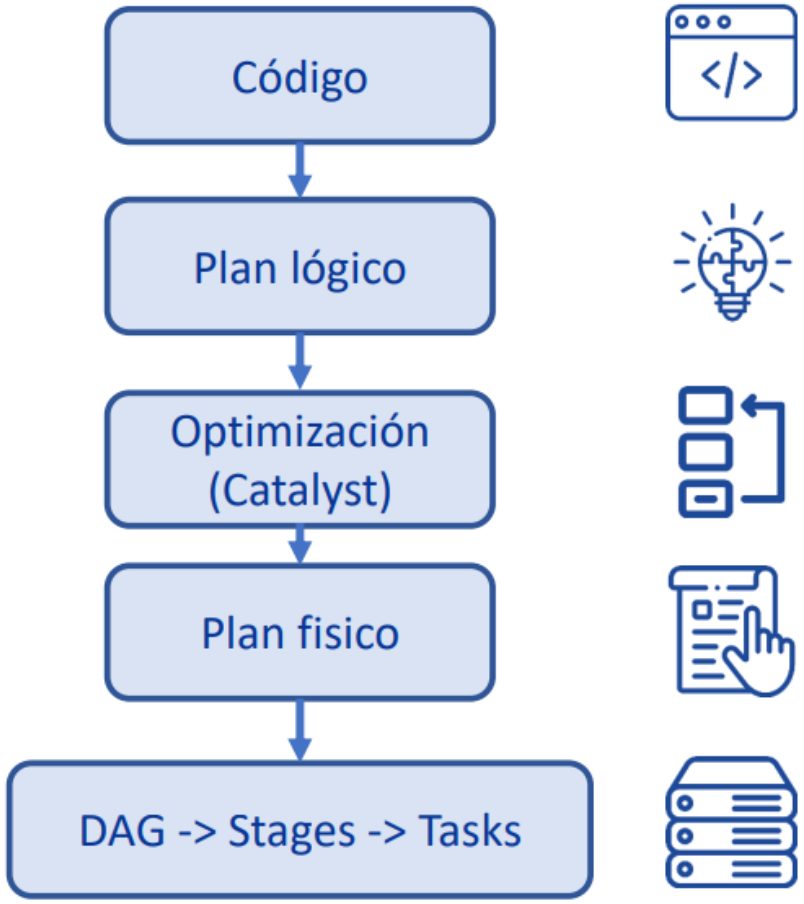
- Las operaciones se completan totalmente o fallan sin dejar cambios
- Los datos siempre cumplen con el esquema definido
- Escrituras concurrentes no interfieren entre sí
- Los datos persisten tras fallos del sistema

Ciclo de ejecución | Spark planifica primero y ejecuta después

Transformaciones (lazy)
Generan plan lógico
select, filter, join

Acciones
Disparan ejecución real
count, show, write

Spark no ejecuta hasta encontrar una acción



Schema Enforcement| Carga datos correctos o no los cargues

Enforcement

Si cargas datos que no cumplen el esquema esperado, el proceso falla (protección automática)

Evolution

Se puede permitir que el esquema evolucione automáticamente si queremos mediante (*mergeSchema*)

Time Travel| Versionado inteligente para tus tablas

Delta Lake guarda un log de transacciones que registra todos los cambios permitiendo consultar versiones anteriores de una tabla

Auditar cambios

Recuperar datos eliminados

Comparar evolución de datos

Optimización de datos| La clave para consultas ultrarrápidas

Optimizer

Compacta pequeños archivos Parquet en archivos grandes

mejora el rendimiento de lectura.

Z - Ordering

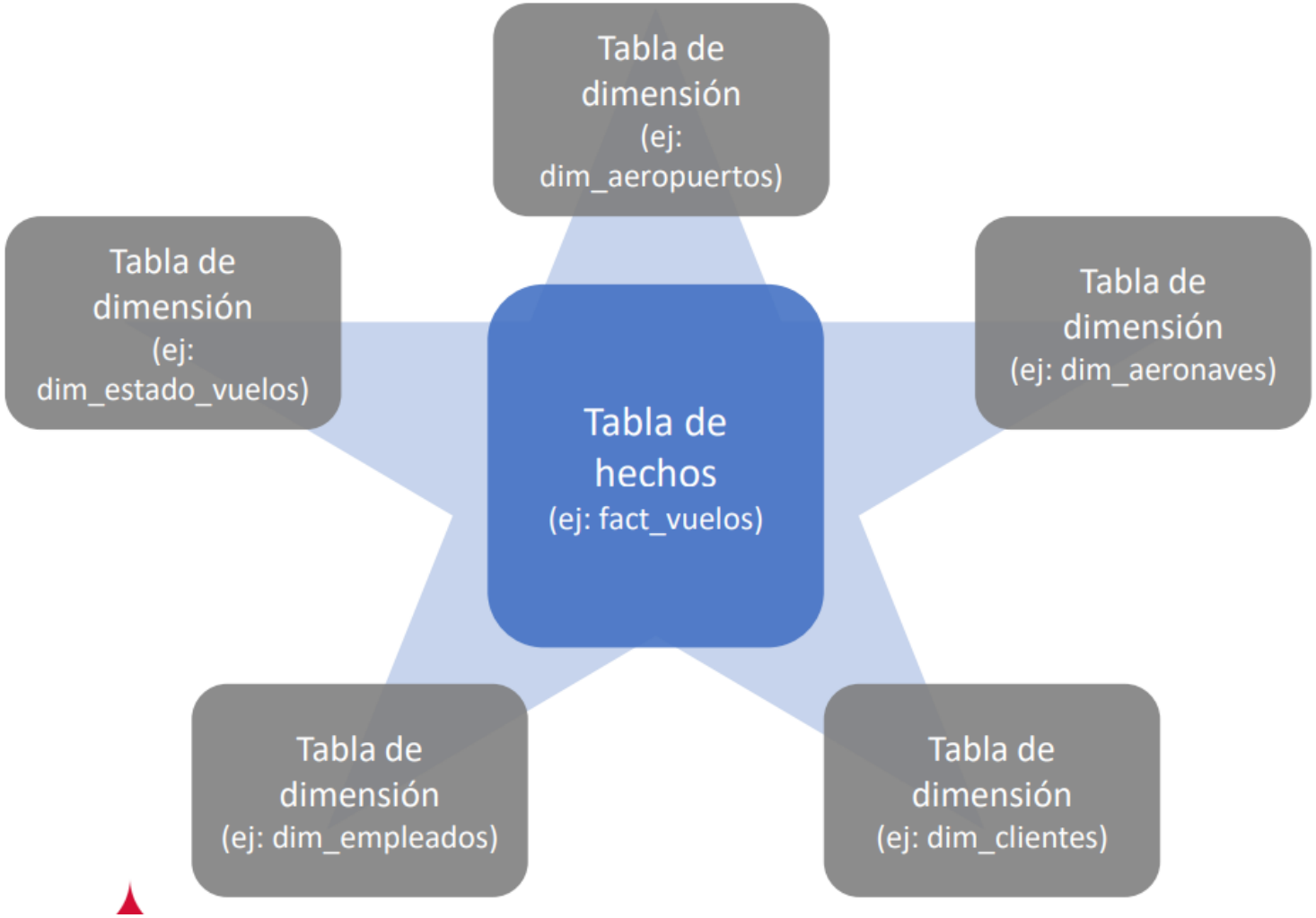
Ordena los datos físicamente por una o más columnas

mejora búsquedas selectivas.

Pipelines ETLs | Del dato crudo al valor con eficiencia

Capa	Staging / Bronce		Core / Silver	Presentation / Gold	
Sublayer	Raw	Standard	Conformed	Datamart	Report
Propósito	• Recogida de las fuentes de datos • Forma original • Auditoría y recuperación de errores	• Datos limpios • Preparación inicial para análisis	• Entidades de negocio (cliente, producto, transacción, ...) • Datos enriquecidos • Agregaciones o cálculos	• Diseñada para ML • Variables listas para entrenamiento	• Listo para aplicaciones finales • Necesidades de negoco
Consumidor	Data Engineering	Data analyst, Engineers	Data Engineers, analyst Scientist	Data Scientist, BI	"C" Roles

Pipelines ETLs | Del dato crudo al valor con eficiencia



Pipelines ETLs | Del dato crudo al valor con eficiencia

Job

Permite lanzar notebooks, scripts o pipelines de forma automática

Workflow

Interfaz para construir, visualizar y gestionar pipelines complejos formados por múltiples tareas encadenadas

Buenas prácticas en Producción| Puntos a tener en cuenta para mantener en un entorno en perfecto estado

Separación de entornos

- DEV / STAGGING / PROD
- Mediante Workspace separados

Configuraciones externas

- Ficheros configs
- Variables entorno o secret

Uso eficiente de clústeres

- Job clusters
- Autoscaling

Manejo de errores

- Reintentos automáticos
- Logging

Versionado

- Repos

DEMO

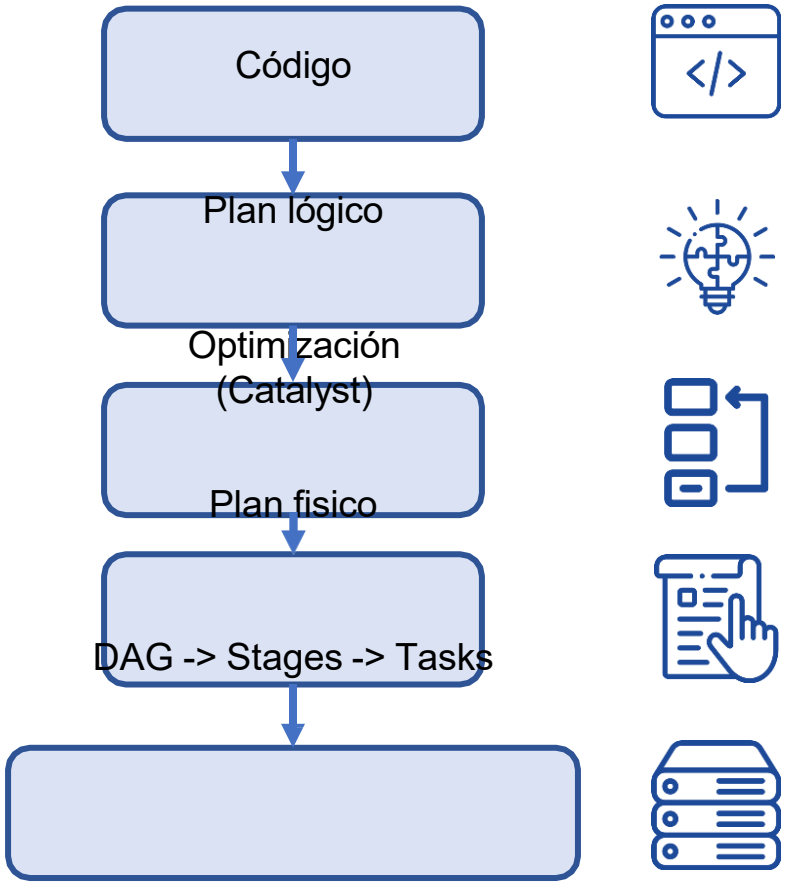
CONSTRUIR UN WORKFLOW EN PRODUCCIÓN

Ciclo de ejecución | Spark planifica primero y ejecuta después

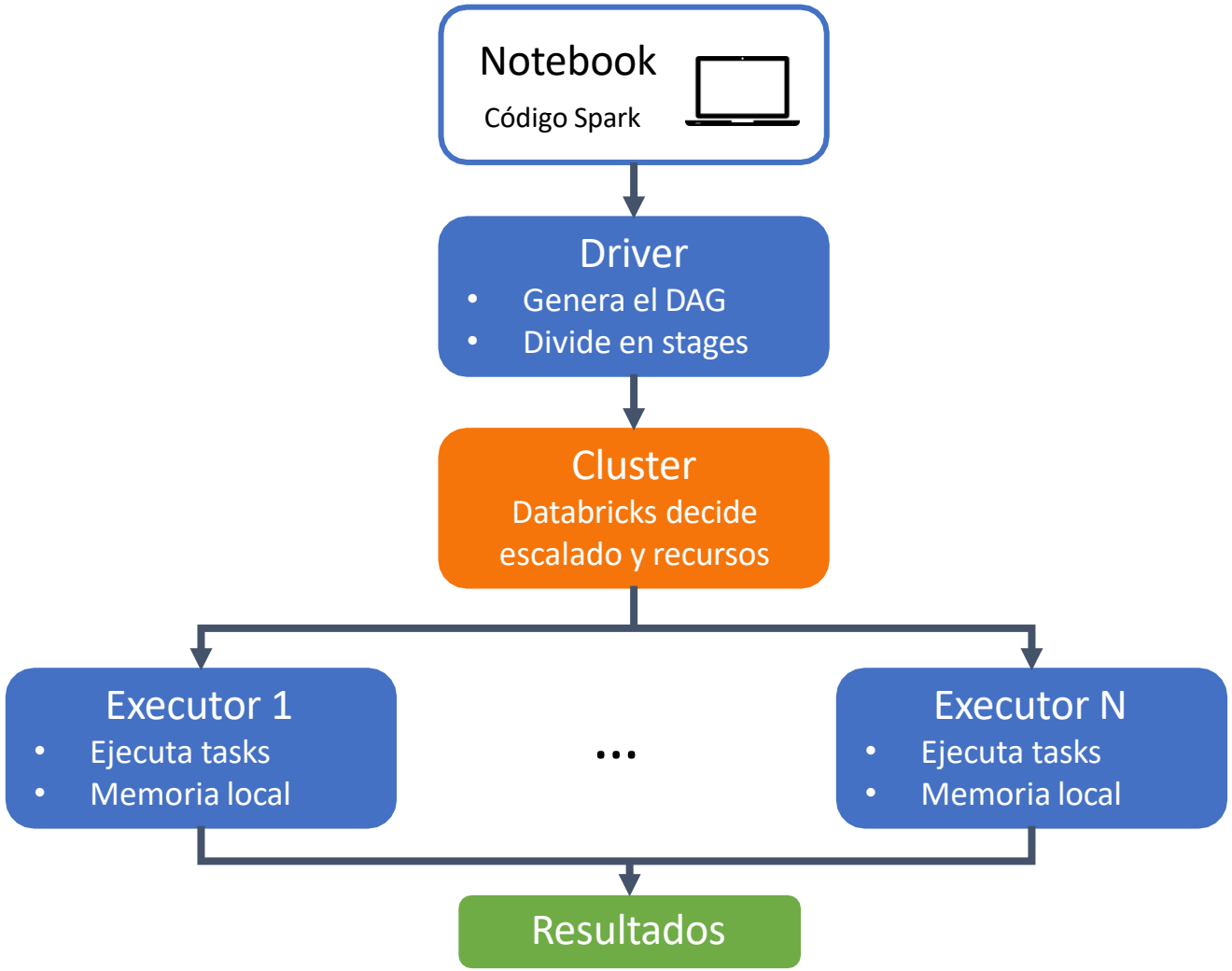
Transformaciones (lazy)
Generan plan lógico
select, filter, join

Acciones
Disparan ejecución real
count, show, write

Spark no ejecuta hasta encontrar una acción



Spark y Databrick por dentro | Spark reparte el trabajo y Databricks gestiona automáticamente



Cuellos de botella | Detectarlos es el primer paso para optimizar un pipeline

Shuffles

- Redistribución de datos entre nodos
- Causas: join, groupBy,…
- Impacto: alto coste

Spark UI

Data Skew

- Particiones con datos muy desbalanceados
- Una tarea tarda mucho más que el resto

DAG Viewer

Particionado ineficiente

- Pocas particiones implica poco paralelismo
- Demasiadas particiones implica sobrecarga de tareas

explain()
queryExecution

Como Diagnosticar En Spark

Concepto	Qué es	Cuándo se crea	Nivel
Job	Acción completa *count, save, collect*	Se ejecuta una acción	Puede contener varios stages
Stage	Operaciones que pueden ejecutarse sin mover datos	Cada operación que requiere shuffle *groupby, join, repartition*	Contiene varios task
Task	Unidad indivisible de ejecución	Por partición de los datos	Se ejecutan en pararlelos los executors

Ejemplos

Ejemplo 1 | Un solo Job, un solo Stage

```
df = spark.range(0, 10)
df.count()
```

- **Jobs:** 1 -> count es acción
- **Stages:** 1 -> No hay suffle (solo lectura)
- **Tasks:** El número de particiones del DF (el que haya por defecto)

Ejemplo 2 | Un solo Job, varios Stages

```
df = spark.range(0, 10_000).repartition(4)
df.groupBy().sum().collect()
```

- **Jobs:** 1 -> collect es acción
- **Stages:** 2 -> groupby require shuffle
- **Tasks:** 8 -> Cada stage tiene 4 Task

Ejemplos

Ejemplo 3 | Varios Jobs

```
df = spark.range(0, 100)

df.count()          # Primer Job
df.filter("id < 50").count()  # Segundo Job
```

- **Jobs:** 2 -> cada acción dispara un nuevo Job
- **Stages:** 2 (1 por cada job)
- **Tasks:** El número de particiones del DF (el que haya por defecto)

Ejemplo 4 | Join con shuffle

```
df1 = spark.range(0, 100).withColumnRenamed("id", "key")
df2 = spark.range(50, 150).withColumnRenamed("id", "key")

df1.join(df2, "key").count()
```

- **Jobs:** 1 -> count es acción
- **Stages:** 2 -> join implica shuffle
- **Tasks:** El número de particiones del DF (el que haya por defecto)

Buenas prácticas de código | Selecciona lo mínimo, filtra pronto y une con cuidado

Selección de columnas

- Evitar *select* *

Filtros tempranos

- Filtros lo antes posibles
- Spark delega a la fuente de datos

Joins eficientes

- Broadcast join solo en tablas pequeñas
- Repartition antes del join

Caché

- Solo si el df se reutiliza

Optimización de particionado y tamaño de archivos| Ni demasiado grande ni demasiado fragmentado

Particionamiento en tablas Delta

- Solo por columnas de alta cardinalidad
- Con filtros frecuentes
- *year, month, country*

Compactación

- Ajuste según tamaño del dataset y cluster
- Compactar archivos pequeños
- *Optimize* o *coalesce/repartition*

Optimización en Databricks | Delta Tables da herramientas para mantener el rendimiento de las tablas a lo largo del tiempo

Z-Ordering

- Reordena físicamente los datos con alta cardinalidad y filtros recurrentes

Optimize

- Combina pequeños archivos en archivos más grandes

Vacuum

- Limpia archivos no referenciados

Optimización en Databricks | Databricks automatiza muchas optimizaciones

AQE (Adaptative Query Execution)

- Ajusta particiones y planes de ejecución en tiempo de ejecución

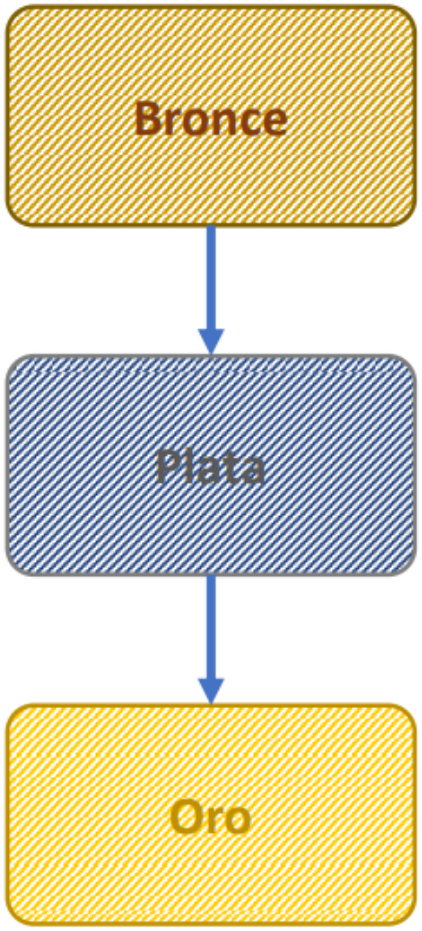
Photo Engine

- Acelera operaciones SQL y DF con un motor en C++

Autoscaling y auto-termination

- Añade o quita nodos según la carga

Buenas prácticas | Modular, parametrizable y resiliente a errores



- Evitar hardcodeos
- Usar ficheros JSON/YAML

- Manejo try/except
- Reintentos configurables en Databricks
- Alertas

Control de costes | Un pipeline sostenible no solo se optimiza en rendimiento, también en coste de ejecución.

Elección cluster

- All-purpose
- Job cluster
- Clusters con Photon

Reducción tiempos

- Auto-termination
- Evitar sin uso

Reutilización

- Job diferentes comparten clusters
- Pools de instancias